

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA Nº10

NOMBRE COMPLETO: Araiza Valdés Diego Antonio

Nº de Cuenta: 423032833

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE: 2026-1

FECHA DE ENTREGA LÍMITE: 09/11/2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1. Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.
   1.1. Ejercicio 1: Animar algún objeto por medio de Keyframes (que no sea ninguno de los personajes principales ni humanoide) requiere compartir capturas de pantalla donde se vea la impresión de los keyframes guardados o reproducidos en consola y guardados en un archivo de tipo txt y compartir los keyframes para que yo pueda reproducir su animación.



Primero buscamos, texturizamos y acomodamos un modelo de una campana, y lo exportamos como obj.

```
Model bell;
```

```
bell = Model();
bell.LoadModel("Models/Bell.obj");
```

Posteriormente cargamos la campana al programa

```
// Variables para animación de campana
float posXcampana = -5.0f, posYcampana = 5.0f, posZcampana = 0.0f;
float rotCampana_z = 0.0f;  // Rotación en Z (balanceo izquierda-derecha)


#define MAX_FRAMES 100 //Número de cuadros máximos
int i_max_steps = 100; //Número de pasos entre cuadros para interpolación
int i_curr_steps = 0;
typedef struct _frame
{
    /* ... */
    // Variables para campana
    float rotCampana_z;     //Variable para RotacionZ de campana
    float rotCampana_zInc;  //Variable para IncrementoRotacionZ

}FRAME;
```

Luego se declararon e inicializaron las variables de la posición y la rotación de la campana y dentro de la estructura frame declaramos las variables para la rotación y su incremento.

```
void saveFrame(void) //tecla L
{
    if (FrameIndex >= MAX_FRAMES) {
        printf("Error: Limite de frames alcanzado\n");
        return;
    }
    printf("frameindex %d\n", FrameIndex);

    /* ... */
    // Guardar rotaciones de campana
    KeyFrame[FrameIndex].rotCampana_z = rotCampana_z;

    //Se agregan nuevas líneas para guardar más variables si es necesario
    //no volatil,se requiere agregar una forma de escribir a un archivo para guardar los frames
    std::ofstream file("Keyframes.txt", std::ios::app);
    if (file.is_open())
    {
        /* ... */

        file << rotCampana_z << "\n";
        file.close();
        printf("Keyframe guardado");
        FrameIndex++;
    }
    else
    {
        printf("No se pudo abrir el archivo\n");
    }
}
```

```
void loadFrames(void) // Cargar keyframes desde archivo al inicio
{
    std::ifstream file("Keyframes.txt");
    if (file.is_open())
    {

        FrameIndex = 0;
        /* ... */
        float campZ;
        while (file >> campZ && FrameIndex < MAX_FRAMES)
        {
            KeyFrame[FrameIndex].rotCampana_z = campZ;
            FrameIndex++;
        }
        file.close();
        printf("Keyframes cargados: %d frames\n", FrameIndex);
    }
    else
    {
        printf("No se encontro el archivo\n");
    }
}

void resetElements(void) //Tecla 0
{
    /* ... */
    rotCampana_z = KeyFrame[0].rotCampana_z;
}
```

```
void interpolation(void)
{
    /* ... */
    // Interpolación para rotaciones de campana
    KeyFrame[playIndex].rotCampana_zInc = (KeyFrame[playIndex + 1].rotCampana_z - KeyFrame[playIndex].rotCampana_z) / i_max_steps;
}
```

Luego, se modificaron las funciones de guardar y cargar keyframes y las funciones de reset e interpolación.

```
rotCampana_z += KeyFrame[playIndex].rotCampana_zInc;
i_curr_steps++;
```

Además, se modificó la función anímate para que solo afectase la rotación en z de la campana.

```
// Campana
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(posXcampana, posYcampana, posZcampana));
model = glm::rotate(model, rotCampana_z * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
bell.RenderModel();
```

Posteriormente se instanció, se asignó la traslación declarada anteriormente, y la rotación de la animación. Luego se renderizó

```
if (keys[GLFW_KEY_L])
{
    if (guardoFrame < 1)
    {
        saveFrame();
        printf("\nrotCampana_z es: %f\n", rotCampana_z);
        printf("\npresiona P para habilitar guardar otro frame");
        guardoFrame++;
        reinicioFrame = 0;
    }
}
if (keys[GLFW_KEY_K])
{
    if (ciclo10 < 1)
    {
        Zrot = !Zrot;
        if (Zrot)
        {
            printf("\n Rotacion en direccion positiva");
        }
        else
        {
            printf("\n Rotacion en direccion negativa");
        }
        ciclo10++;
        ciclo11 = 0;
    }
}

/* ... */

if (keys[GLFW_KEY_5])
{
    if (ciclo4 < 1)
    {
        if (Zrot)
        {
            rotCampana_z += 10.0f;
        }
        else
        {
            rotCampana_z -= 10.0f;
        }
        printf("\n rotCampana_z es: %f", rotCampana_z);
        ciclo4++;
        ciclo5 = 0;
        printf("\n Presiona la tecla 6 para poder habilitar la variable");
    }

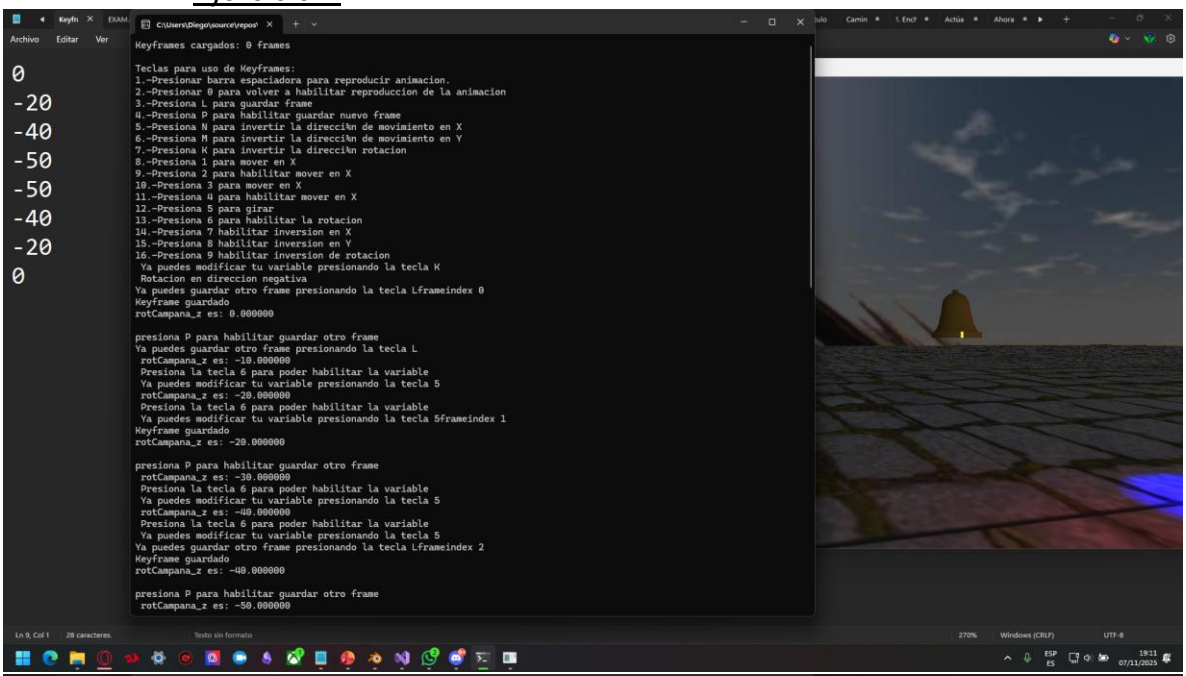
}
if (keys[GLFW_KEY_6])
{
    if (ciclo5 < 1)
    {
        ciclo4 = 0;
        ciclo5++;
        printf("\n Ya puedes modificar tu variable presionando la tecla 5");
    }
}

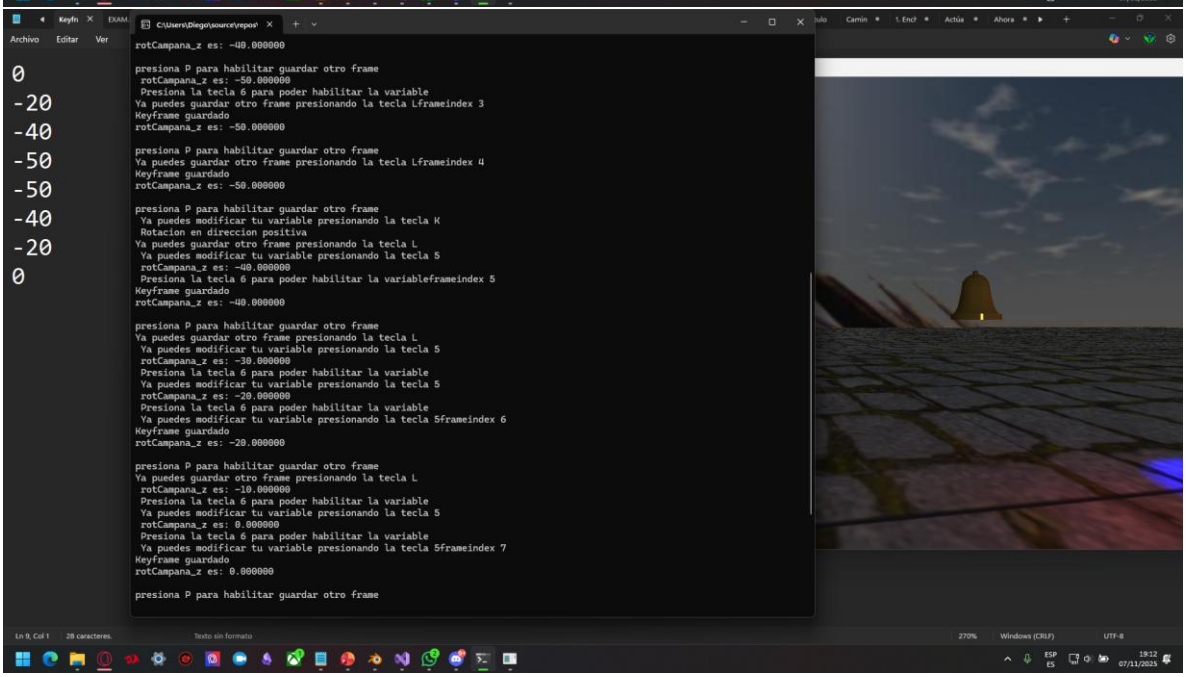
if (keys[GLFW_KEY_9])
{
    if (ciclo11 < 1)
    {
        ciclo10 = 0;
        ciclo11++;
        printf("\n Ya puedes modificar tu variable presionando la tecla K");
    }
}
```

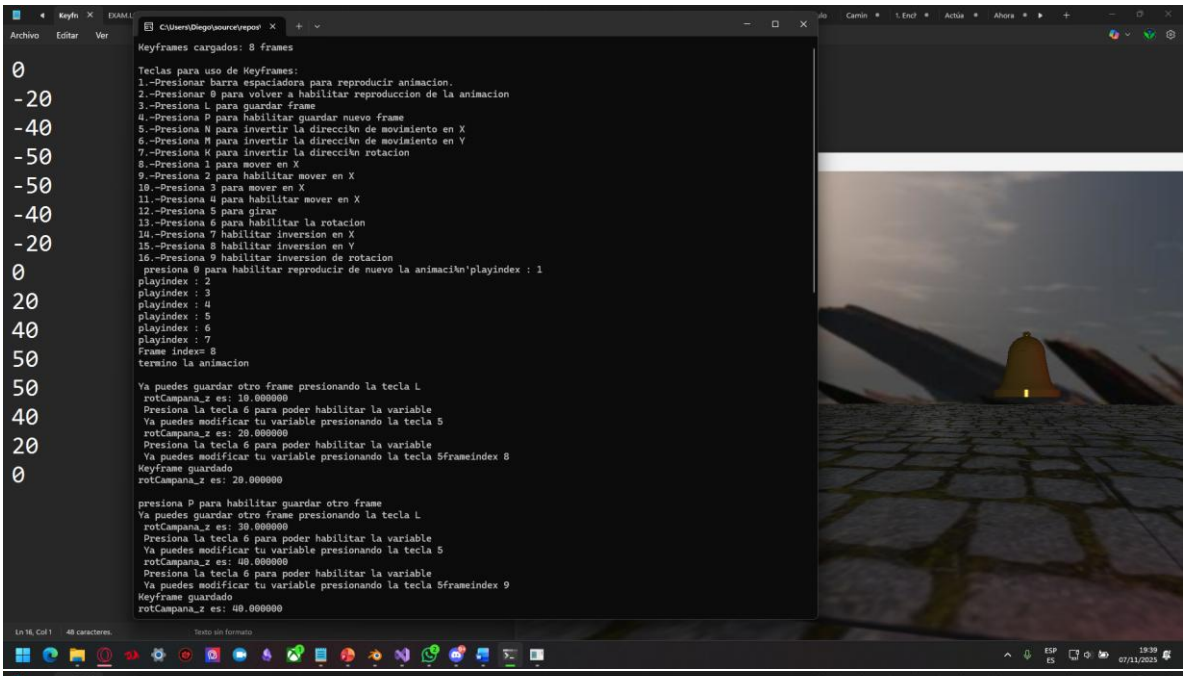
Finalmente, dentro de inputKeyframes se adaptaron las funciones ya existentes para modificar la rotación y se comentaron las demás.

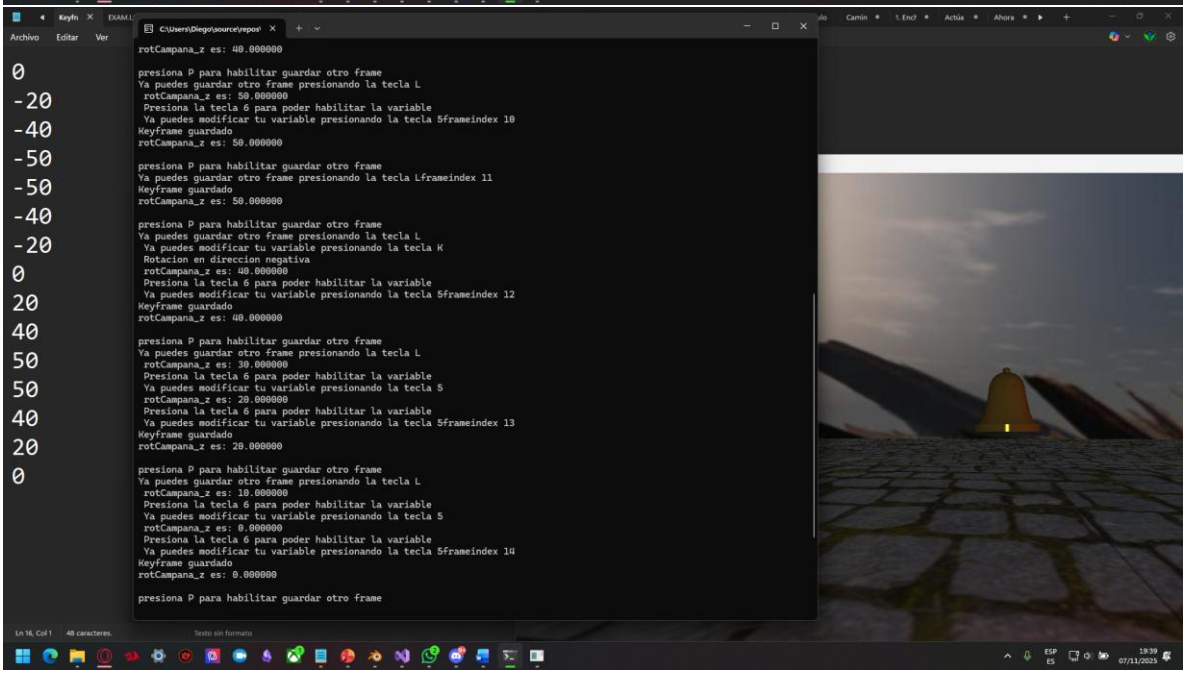
Evidencias:

1.1.1. Ejercicio 1









Enlace al video:

https://drive.google.com/file/d/1NYDL-YUgeztggwuRkb7z4KNzpuO8NQpr/view?usp=sharing

1. Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla
   R: No hubo problemas.

2. Conclusión:
   1. Los ejercicios del reporte: Complejidad, Explicación.
      Respecto a los ejercicios realizados, fueron bastante adecuados a lo visto en clase y con los ejemplos vistos, no hubo mayor problema.

   2. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica.
      La explicación fue muy buena, se entendió perfectamente como animar objetos y que es y que no es animación. Sin embargo, me hubiera gustado que explique un poco más la lógica acerca de la siguiente línea:
      glUniform2fv(uniformTextureOffset, 1, glm::value_ptr(toffset));
      ya que, si bien la entendí, solo fue muy por encima.

   3. Conclusión
      Personalmente creo que se cumplieron los objetivos de la práctica. Con el ejercicio y la práctica, aprendí a crear diversas animaciones haciendo uso de banderas, condicionales y funciones, como movimiento (traslación y rotación), cambiar texturas y hacer letreros movibles (con y sin pausas visibles). Además, aprendí a utilizar la variable deltaTime.